

REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR

Université de Yaoundé I

Institut Saint Jean (ISJ)

REPUBLIC OF CAMEROON

Peace – Work – Fatherland

MINISTRY OF HIGHER EDUCATION

University of Yaounde I

Institut Saint Jean (ISJ)

CAHIER TECHNIQUE D'IMPLEMENTATION

CONCEPTION, DEVELOPPEMENT ET DEPLOIEMENT D'UN WEB-BASED LINUX NAMESPACE ISOLATION MANAGER

Gestionnaire d'isolation de processus Linux via interface web

Rédigé et présenté par :

➔ *TCHINDA Florelle Ecladore*

➔ *PAYONG Brice Valery*

➔ *ZOUA HOULI Abraham*

Sous la supervision de M. NGUIMBUS Emmanuel

Année Académique 2025-2026

SOMMAIRE

1. Introduction et objet du document

Rôle de ce document. Le Cahier d'analyse a défini ce que le système devait faire, et le Cahier de conception a traduit ces besoins en structures (classes, séquences, architecture 3-tier). Le présent Cahier Technique d'Implémentation ferme la boucle : il explique comment le système a réellement été construit, module par module, en partant du clic dans le navigateur jusqu'au processus Linux véritablement isolé sur la machine hôte.

Ce document est rédigé dans un objectif pédagogique : chaque terme technique (namespace, capability, PAM, JWT, WebSocket...) est expliqué à sa première apparition, afin qu'il puisse être lu et compris indépendamment des deux cahiers précédents, tout en restant cohérent avec eux.

Ce que ce document couvre : l'architecture réellement mise en œuvre, le modèle de données SQLite, le module d'authentification adossé aux comptes Linux réels, le module de gestion des sandboxes, le terminal web temps réel, le programme « helper » en C qui constitue le cœur de l'isolation, la configuration des droits système, la distinction entre les modes ATTACK et DEFENSE, ainsi que le déroulement complet d'une requête de bout en bout.

Principe directeur du projet —

*Aucun sudo, aucun mot de passe applicatif stocké en base, aucun binaire SUID root.
Tout privilège système est encapsulé dans un unique petit programme C porteur de
capabilities Linux ciblées : c'est la pierre angulaire de toute la sécurité de l'application.*

2. Architecture générale

2.1 Trois couches, une seule responsabilité chacune

L'application conserve l'architecture en couches définie au stade de la conception, mais l'implémentation ajoute une pièce essentielle non anticipée dans le détail : un helper natif écrit en C, interposé entre le Backend Flask et le noyau Linux.

Couche	Rôle / technologie	Niveau de privilège
1. Frontend	React + Vite, exécuté dans le navigateur de l'utilisateur	Aucun — HTML/JS pur, ne fait qu'appeler des URLs et afficher les réponses
2. Backend	Flask (Python), exécuté sous l'utilisateur Linux « sandboxmgr »	Non-root — connaît la base de données, les JWT, la logique métier, mais ne peut ni créer de namespace ni faire <code>setuid()</code>
3. Helper C	Binaire natif /opt/sandboxmgr/helper/sandbox_helper	Porteur de capacités Linux ciblées (<code>cap_setuid</code> , <code>cap_sys_admin</code> , ...) — seule pièce autorisée à toucher au noyau pour les opérations sensibles

2.2 Le trajet d'une action utilisateur

Une action déclenchée dans le navigateur (par exemple « créer une sandbox ») descend ainsi à travers les couches :

- Le **Frontend** (`src/lib/api.ts`, `src/lib/auth.tsx`, `src/hooks/useSandboxTerminal.ts`) envoie une requête HTTPS ou un message WebSocket, en y attachant systématiquement le jeton JWT de session.
- Le **Backend Flask** (`app/auth`, `app/sandboxes`, `app/ws`, `app/privilege`, `app/models`) authentifie la requête, applique la logique métier, interroge ou met à jour la base SQLite, puis délègue toute opération privilégiée au helper via `subprocess.run(...)`.
- Le **Helper C** reçoit une sous-commande stricte (`useradd`, `spawn`, `exec`, `kill`), valide ses arguments, et parle directement au noyau (`clone()`, `setns()`, `mount()`, `setuid()`...).
- Le **noyau Linux** exécute réellement l'isolation par namespaces et renvoie le résultat, qui remonte ensuite les trois couches en sens inverse jusqu'à l'affichage dans le navigateur.

2.3 Pourquoi couper le privilège en trois ?

Cette séparation applique strictement le principe du moindre privilège. Le frontend n'a aucun privilège. Le backend Flask tourne en tant qu'utilisateur non-root : même en cas de faille (injection, désérialisation, etc.), un attaquant qui compromettrait le processus Flask n'obtiendrait pas plus que les quatre opérations strictement définies que le helper accepte de réaliser — pas un accès root arbitraire à la machine.

Le helper C, lui, est volontairement minuscule (environ 300 lignes), auditable dans son intégralité, et valide strictement chacun de ses arguments avant d'agir. C'est la seule pièce du système porteuse de capacités Linux.

3. Modèle de données

La persistance repose sur trois tables SQLite, définies dans app/models/__init__.py avec SQLAlchemy comme ORM.

3.1 Table « Utilisateur »

Champ	Type	Rôle
id	UUID	Identifiant applicatif ; placé dans le JWT sous la clé « sub »
nom_systeme	string	Nom du compte Linux réel (« alice », « bob »...)
uid_systeme	int	UID Linux (ex. 1001), utilisé pour le setuid() lors de l'exécution dans une sandbox
date_derniere_connexion	datetime	Mise à jour à chaque connexion réussie
Point de sécurité essentiel <i>Aucun mot de passe n'est stocké dans cette table. L'authentification réelle est intégralement déléguée à PAM, qui consulte /etc/shadow, sous la responsabilité du système d'exploitation hôte.</i>		

3.2 Table « Sandbox »

Champ	Type	Rôle
id	UUID	Identifiant applicatif
nom_virtuel	string	Nom lisible choisi par l'utilisateur (« ma-bulle »)
pid_racine	int	PID hôte du processus « init » de la sandbox, retourné par clone()
statut	enum	EN_COURS, ARRETEE, EN_ERREUR
proprietaire_id	UUID	Clé étrangère vers Utilisateur.id

Le champ pid_racine est la clé qui relie la ligne de la base au processus réellement vivant dans le noyau : c'est via /proc/<pid_racine>/ns/* que le helper rejoint ensuite les namespaces de cette sandbox lors d'un exec.

3.3 Table « Commande »

Un journal en ajout seul (append-only) : chaque commande tapée dans le terminal produit une ligne avec texte_instruction, resultat_sortie, est_reussie et date_execution. Cette table alimente l'onglet « Historique » du frontend et sert de preuve d'exécution lors de la démonstration.

Relations : un Utilisateur possède 0 à N Sandbox ; chaque Sandbox conserve l'historique de 0 à N Commande, avec suppression en cascade (cascade="all, delete-orphan") lorsque la sandbox est détruite.

4. Module d'authentification

4.1 Deux principes fondateurs

- **1. Ne jamais réinventer la gestion des mots de passe.** L'application délègue à PAM (Pluggable Authentication Modules), la brique standard de Linux qui compare un mot de passe saisi au hash stocké dans /etc/shadow. Conséquence directe : même politique de mots de passe que le système, même gestion du verrouillage après échecs répétés, et aucun secret applicatif supplémentaire à protéger.
- **2. Émettre un JWT après connexion.** Un JSON Web Token est une chaîne signée contenant l'identifiant utilisateur et son UID Linux. Le frontend le conserve dans localStorage et le joint à chaque requête via l'en-tête HTTP « Authorization: Bearer <jwt> ».

4.2 Côté frontend

- src/lib/api.ts : la fonction login(username, password) appelle POST /api/auth/login, reçoit { token, user }, que l'AuthProvider enregistre en localStorage.
- src/lib/auth.tsx : expose le contexte React AuthProvider et le hook useAuth() (user, token, login(), logout()). Sur logout(), le localStorage est purgé et tout l'état React est remis à zéro, si bien que les sandboxes de l'utilisateur précédent disparaissent immédiatement de l'écran.
- Toute réponse 401 déclenche automatiquement la purge de la session et une redirection vers l'écran de connexion, ce qui évite tout affichage fantôme après expiration du jeton.

4.3 Inscription — POST /api/auth/register

Séquence exacte suivie par le backend lors de la création d'un compte :

```
utilisateur envoie { username, password }
|
Flask valide le format (username alphanumérique 2-32, password >= 8)
|
privilege_manager.create_linux_user(username, password)
|
subprocess.run([helper, "useradd", username], input=password)
|
helper : execl("/usr/sbin/useradd", "-m", "-s", "/bin/bash", username)
        puis execl("/usr/sbin/chpasswd") pour poser le mot de passe
|
Flask lit l'UID via getpwnam(username), insère l'utilisateur en base
```

Le mot de passe n'est jamais journalisé ni stocké : il transite uniquement sur l'entrée standard (stdin) d'un sous-processus, puis est oublié.

4.4 Connexion — POST /api/auth/login

```
if not pam.pam().authenticate(username, password, service="login"):
    return 401
uid = get_uid(username)          # via getpwnam
user = get_or_create_utilisateur(...) # miroir applicatif
token = jwt.encode({"sub": user.id, "uidSysteme": uid, "exp": now+8h}, SECRET)
```


Le JWT embarque directement uidSysteme : lorsque l'utilisateur exécutera plus tard une commande, cet UID est lu tel quel dans le token, sans avoir besoin de reconsulter /etc/passwd à chaque appel.

4.5 Le décorateur @login_required

Chaque route protégée applique la même vérification :

```
token = request.headers["Authorization"][7:]      # apres "Bearer "  
claims = jwt.decode(token, JWT_SECRET, ["HS256"]) # verifie signature + expiration  
g.user_id = claims["sub"]  
g.user_uid = claims["uidSysteme"]  
g.user_name = claims["nomSysteme"]
```

Un jeton expiré ou falsifié fait échouer le décodage et provoque une réponse 401.

5. Module de gestion des sandboxes

Le blueprint `app/sandboxes/routes.py` implémente trois opérations — créer, lister, supprimer — en une centaine de lignes très linéaires.

5.1 Création — POST `/api/sandboxes`

- Le frontend appelle `createSandbox("ma-bulle")` depuis `src/lib/api.ts`.
- Flask crée une ligne `Sandbox` en statut `ARRETEE` (`db.session.flush()`) : la ligne existe en mémoire, sans être encore validée en base).
- Flask appelle `privilege_manager.spawn_sandbox(mode, uid, hostname)`, qui invoque le helper : `subprocess.run([helper, "spawn", mode, uid, hostname])`.
- Le helper effectue un `clone()` qui crée un véritable processus `init` dans les nouveaux namespaces, puis imprime « `PID <n>` » sur `stdout` avant de se terminer. L'`init`, lui, reste vivant et devient `PID 1` dans son propre namespace `PID`.
- Flask récupère ce `PID`, écrit `pid_racine`, bascule le statut à `EN_COURS` et valide (`commit`).
- La réponse renvoyée est `201` avec l'objet `sandbox` créé.

Gestion des échecs —

Si le helper échoue à n'importe quelle étape, la ligne est annulée (rollback) : aucune sandbox « fantôme » ne peut apparaître en base sans processus réel associé.

5.2 Listing — GET `/api/sandboxes`

```
items = Sandbox.query.filter_by(proprietaire_id=g.user_id).all()
```

Le filtrage est appliqué exclusivement côté serveur — le frontend ne refiltre jamais les résultats. Ainsi, même en forgeant manuellement une requête, un jeton `JWT` ne peut jamais faire remonter que les sandboxes de son propre propriétaire.

5.3 Suppression — DELETE `/api/sandboxes/:id`

- Vérification que l'utilisateur courant est bien le propriétaire (sinon réponse `404`, pour ne pas révéler l'existence d'une sandbox appartenant à quelqu'un d'autre).
- Appel à `kill_sandbox(pid_racine)`, qui invoque le helper `kill`, lequel envoie un `SIGTERM` au processus `init`. Le noyau nettoie alors automatiquement tous les namespaces dès que le dernier processus qui les référence disparaît.
- `db.session.delete(sb)` puis `commit()` — la relation `cascade="all, delete-orphan"` sur les commandes supprime l'historique lié dans la même opération.

6. Exécution des commandes — le terminal WebSocket

6.1 Pourquoi un WebSocket plutôt que du HTTP classique ?

Un terminal se juge à sa latence perçue : l'utilisateur veut voir la sortie s'afficher au fil de l'eau. Une requête HTTP classique paierait le coût d'un nouvel établissement de connexion à chaque commande. Le WebSocket ouvre un tuyau bidirectionnel une seule fois, réutilisé pour toutes les commandes de toute la session.

6.2 Ouverture de la connexion

Frontend (src/hooks/useSandboxTerminal.ts) :

```
const url = `${WS_BASE_URL}/ws/sandboxes/${sandboxId}/terminal?token=${jwt}`;  
const ws = new WebSocket(url);
```

Le jeton JWT est transmis en paramètre d'URL car l'API navigateur `new WebSocket(...)` ne permet pas d'ajouter un en-tête HTTP personnalisé. Une reconnexion automatique est prévue, avec un délai croissant (1 s → 2 s → 5 s puis abandon).

Backend (app/ws/terminal.py) :

```
@sock.route("/ws/sandboxes/<sandbox_id>/terminal")  
def terminal(ws, sandbox_id):  
    uid, user_id = _auth(ws)          # decode le JWT depuis ?token=  
    sb = Sandbox.query.get(sandbox_id)  
    if sb.proprietaire_id != user_id: return # 403 silencieux
```

6.3 Protocole applicatif (JSON sur WebSocket)

Sens	Payload	Rôle
client -> serveur	<code>{"type":"cmd","text":"ls -la"}</code>	Commande à exécuter
client -> serveur	<code>{"type":"history_prev"}</code>	Flèche haut dans le terminal
serveur -> client	<code>{"type":"stdout","text":"..."}</code>	Sortie de la commande
serveur -> client	<code>{"type":"exit","ok":true}</code>	Fin de commande
serveur -> client	<code>{"type":"history","text":"ls"}</code>	Réponse à history_prev
serveur -> client	<code>{"type":"error","message":"..."}</code>	Erreur métier

6.4 Traitement côté serveur (mode DEFENSE)

```
while True:  
    raw = ws.receive()
```

```
msg = json.loads(raw)
if msg["type"] == "cmd":
    argv = shlex.split(msg["text"])    # split shell-safe
    if argv[0] not in WHITELIST:      # ls, pwd, whoami, cat, ...
        ws.send({"type": "error", "message": "command_not_allowed"})
        continue
    out, ok = exec_in_sandbox(uid, sb.pid_racine, argv)
    db.session.add(Commande(texte=..., resultat=out, est_reussie=ok, ...))
    db.session.commit()
    ws.send({"type": "stdout", "text": out})
    ws.send({"type": "exit", "ok": ok})
```

- **shlex.split** analyse la ligne comme le ferait un shell, mais sans jamais exécuter de shell : pas d'expansion de variables, et aucune injection possible via un caractère comme « ; » suivi d'une autre commande.
- **Liste blanche stricte** en mode DEFENSE : seule une petite liste de commandes (ls, pwd, whoami, id, cat, echo, hostname, ps, uname, date, env) est autorisée. Toute autre commande est rejetée avant même d'atteindre le helper.

En mode ATTACK, cette même étape est remplacée par `argv = ["/bin/bash", "-c", text]` — volontairement vulnérable, dans un but strictement démonstratif encadré (voir section 9).

6.5 exec_in_sandbox côté Python

```
subprocess.run([helper, "exec", str(uid), str(pid_racine), *argv],
               capture_output=True, timeout=30)
```

Toute la mécanique namespace / PID / UID est entièrement déléguée au helper — Flask ne fait ici que transmettre et attendre le résultat.

7. Le helper C — cœur de l'isolation

7.1 Pourquoi un binaire séparé, écrit en C ?

Python ne peut pas porter fiablement une capability Linux : l'interpréteur est partagé, potentiellement chargé de modules tiers — un interpréteur Python en SUID serait un désastre de sécurité. Le helper C, à l'inverse, est un binaire minuscule et isolé, auquel on attribue exactement les droits nécessaires via la commande setcap. C'est l'unique chemin d'élévation de privilège de toute l'application.

7.2 Les quatre sous-commandes acceptées

Sous-commande	Rôle	Retour
useradd USER	Crée un compte Linux, pose le mot de passe (lu sur stdin)	rc=0 en cas de succès
spawn MODE UID HOSTNAME	Crée l'init d'une sandbox dans de nouveaux namespaces	Imprime « PID <n> »
exec UID PID ARGV...	Exécute une commande dans les namespaces de PID	rc = code de sortie de la commande
kill PID	Envoie SIGTERM à l'init de la sandbox	rc=0

7.3 « spawn » — création d'une bulle isolée

C'est l'opération la plus délicate techniquement. Squelette du code :

```
int flags = SIGCHLD
    | CLONE_NEWNS | CLONE_NEWUTS | CLONE_NEWPID
    | CLONE_NEWNET | CLONE_NEWIPC;
if (defense) flags |= CLONE_NEWUSER;

pid_t pid = clone(sandbox_init, stack + STACK_SIZE, flags, &ctx);
```

Chaque option CLONE_NEW* crée un namespace distinct :

Namespace	Ce qu'il isole
NEWNS	Points de montage (/ , /proc...) — la sandbox voit son propre arbre de montage
NEWUTS	Nom d'hôte (sethostname)
NEWPID	Table des processus — l'init cloné devient PID 1 dans la sandbox
NEWNET	Interfaces réseau et tables de routage, isolées de l'hôte
NEWIPC	Files de messages et sémaphores System V
NEWUSER	Table de correspondance UID/GID — un UID interne 0 (root sandbox) peut correspondre à un UID hôte non privilégié

Après le clone(), une course s'engage : le processus fils tourne déjà dans les nouveaux namespaces, mais c'est le parent qui doit écrire les fichiers uid_map / gid_map (le noyau

interdit au fils d'écrire son propre mapping s'il n'a pas la capability requise dans le user namespace parent). Deux tubes (pipes) servent de points de rendez-vous entre parent et fils :

parent	fils (init sandbox)
clone(...)	-> démarre sandbox_init()
	read(ready_r) <- bloque
write /proc/<pid>/setgroups "deny"	
write /proc/<pid>/uid_map "0 UID 1"	
write /proc/<pid>/gid_map "0 UID 1"	
write(ready_w, "1")	<- debloque
	sethostname(...)
	mount("/", MS_PRIVATE)
	umount2("/proc", MNT_DETACH)
	mount("proc", "/proc", "proc", ...)
	write(ack_w, "1")
read(ack_r)	<- attend l'acquittement
printf("PID %d\n", pid)	dup2(devnull, {0,1,2}) <- libere les pipes
exit(0)	for (;;) pause();

Deux détails, corrigés en cours de développement, se sont révélés critiques :

- **dup2(devnull, 0/1/2) dans l'init.** Sans cela, l'init hérite des tubes stdout/stderr du helper : subprocess.run(capture_output=True) côté Flask attend alors indéfiniment une fin de flux (EOF) qui n'arrive jamais, ce qui provoque un blocage de 15 secondes avant expiration du délai.
- **umount2("/proc", MNT_DETACH) avant mount("proc").** Sans ce démontage préalable, le nouveau montage échoue avec l'erreur EBUSY, car /proc est déjà monté dans la vue héritée du parent.

7.4 « exec » — rejoindre une bulle déjà existante

Il s'agit, en substance, d'un équivalent de « docker exec » tenant en une trentaine de lignes de C :

```
const char *order[] = { "ipc", "uts", "net", "pid", "mnt" };
for (i=0; i<5; i++) fds[i] = open("/proc/<sandbox_pid>/ns/<name>", O_RDONLY);
for (i=0; i<5; i++) setns(fds[i], 0);

pid_t child = fork();          // OBLIGATOIRE apres setns(pid)
if (child == 0) {
    umount2("/proc", MNT_DETACH);
    mount("proc", "/proc", "proc", ...);
    struct passwd *pw = getpwuid(uid);
    initgroups(pw->pw_name, pw->pw_gid);
    setgid(pw->pw_gid);
    setuid(uid);               // abandonne les capabilities
    execvp(argv[0], argv);
}
waitpid(child, &st, 0);
return WEXITSTATUS(st);
```

Trois points subtils expliquent des choix qui pourraient sembler arbitraires :

- On ne fait volontairement *pas* de setns() sur le user namespace : les noyaux récents le refusent (EPERM) même avec CAP_SYS_ADMIN si ce user namespace a été créé par

un autre processus. Il est hérité implicitement via les autres namespaces (mnt/pid) — c'est exactement le comportement de Docker et Podman.

- Le `fork()` après `setns()` sur le namespace PID est obligatoire : le noyau n'applique le nouveau namespace PID qu'aux enfants du processus, jamais au processus appelant lui-même.
- Le `setuid(uid)` intervient en tout dernier : cet appel abandonne définitivement toutes les capacités. Le processus qui exécutera ensuite la commande de l'utilisateur ne peut donc plus rien faire de privilégié, même si l'argument fourni est malveillant.

7.5 Validation stricte des entrées du helper

Chaque argument reçu par le helper est validé avant tout usage :

- `username` : motif `[a-z0-9]{2,32}` (validé par `is_valid_username`) ;
- `uid` : entier compris entre 1000 et 65535 — jamais root, jamais un UID système ;
- `mode` : uniquement « attack » ou « defense » ;
- `sandbox_pid` : entier supérieur ou égal à 2.

Aucun argument n'est jamais transmis à un interpréteur shell : toutes les exécutions passent par `execl/execvp` avec des tableaux `argv[]` explicites, ce qui rend toute injection structurellement impossible à ce niveau.

8. Configuration et gestion des droits système

8.1 Trois utilisateurs Linux impliqués

Utilisateur	Rôle	Peut devenir root ?
root	Propriétaire du binaire helper, installe les capacités	Oui
sandboxmgr	Exécute Flask, écoute sur le port applicatif	Non
alice, bob, ...	Utilisateurs finaux, créés via /api/auth/register	Non

Flask ne tourne jamais en root : il est démarré via systemd sous l'utilisateur sandboxmgr (voir flask-backend/systemd/*.service).

8.2 Attribution des capacités sur le helper

Une capability Linux est une sous-permission de root : elle autorise une action précise (par exemple « changer d'UID ») sans accorder l'intégralité des pouvoirs de root. Ces capacités sont attribuées une seule fois, à l'installation :

<pre>sudo chown root:sandboxmgr helper/sandbox_helper sudo chmod 750 helper/sandbox_helper # executable uniquement par # root et le groupe sandboxmgr sudo setcap cap_setuid,cap_setgid,\ cap_sys_admin,cap_sys_chroot,\ cap_net_admin,cap_dac_override,\ cap_chown,cap_fowner+ep helper/sandbox_helper</pre>		
Capability	Ce qu'elle autorise	Utilisée par
cap_setuid	setuid() vers n'importe quel UID	exec
cap_setgid	setgid() et setgroups()	exec
cap_sys_admin	clone(CLONE_NEW*), setns(), mount(), sethostname()	spawn, exec
cap_sys_chroot	chroot() — utile pour un futur pivot_root	spawn (futur)
cap_net_admin	Manipulation des interfaces réseau (futur veth pair)	spawn (futur)
cap_dac_override	Écrire dans /proc/<pid>/uid_map sans en être propriétaire	spawn
cap_chown, cap_fowner	useradd -m crée un répertoire personnel avec chown	useradd

Le suffixe +ep signifie « effective + permitted » : les capacités sont actives dès le démarrage du binaire et restent disponibles lors de l'exécution des programmes fils.

8.3 Pré-requis système

- sandboxmgr doit disposer d'une plage de sous-UID / sous-GID dans /etc/subuid et /etc/subgid, indispensable pour CLONE_NEWUSER :

```
sudo usermod --add-subuids 100000-165535 --add-subgids 100000-165535 sandboxmgr
```

- libpam0g-dev installé, avec le module Python python-pam disponible pour l'authentification.
- SQLite pour la persistance — aucun serveur de base de données supplémentaire à installer ou administrer.

8.4 Variables d'environnement

Backend (.env) :

Variable	Rôle
SANDBOXMGR_MODE	« defense » ou « attack »
FLASK_SECRET_KEY	Secret Flask (sessions / cookies)
JWT_SECRET	Clé HMAC de signature des JWT
HELPER_BIN	Chemin absolu vers sandbox_helper
DATABASE_URL	Ex. sqlite:///sandboxmgr.db
CORS_ORIGINS	Origines autorisées pour le frontend

Frontend (.env.local) :

Variable	Rôle	Défaut
VITE_API_URL	Base URL REST	http://127.0.0.1:5000
VITE_WS_URL	Base URL WebSocket	ws://127.0.0.1:5000

9. Modes ATTACK et DEFENSE

Le même code source alimente deux services systemd distincts, sur deux ports différents, chacun avec sa propre base de données. Le comportement change uniquement selon la valeur de la variable d'environnement SANDBOXMGR_MODE.

Aspect	DEFENSE (port 8443)	ATTACK (port 9443)
User namespace	Activé (CLONE_NEWUSER)	Désactivé
Mapping root sandbox -> hôte	UID 0 interne -> UID utilisateur, unique	(aucun mapping)
Exécution des commandes	execvp(argv[]) — pas de shell	bash -c "<saisie>" (INJECTION)
Liste blanche de commandes	Stricte, appliquée côté serveur	Aucune
Bannière de l'interface	Verte « DEFENSE »	Rouge « ATTACK — DEMO ONLY »
Cadre pédagogique strict <i>Le mode ATTACK n'existe que pour la démonstration académique encadrée : il illustre concrètement pourquoi concaténer une saisie utilisateur dans « bash -c » est dangereux, et pourquoi l'absence de user namespace transforme un root de sandbox en root réel de la machine hôte. Ce mode ne doit jamais être exposé sur un réseau non maîtrisé.</i>		

Chaque commande exécutée dans les deux modes est systématiquement journalisée (horodatage, identifiant de sandbox, résultat), ce qui constitue la preuve d'exploitation présentée lors de la soutenance, puis la preuve que la même tentative échoue proprement une fois les corrections DEFENSE appliquées.

10. Cycle de vie complet d'une requête

Illustrons le cas le plus riche du système : Alice a déjà créé une sandbox, et tape « ls -la » dans le terminal web.

Éta pe	Composant	Ce qui se passe
1	Navigateur (React)	xterm capte la saisie « ls -la\n » ; useSandboxTerminal.sendCommand("ls -la") envoie le message ws.send({'type':"cmd","text":"ls -la"})
2	Flask (app/ws/terminal.py)	Décode le JWT reçu en paramètre d'URL, vérifie sb.proprietaire_id == user_id, découpe la commande avec shlex.split, vérifie que « ls » figure dans la whitelist, puis appelle exec_in_sandbox(uid=1001, pid=26404, ["ls","-la"])
3	Helper C (cmd_exec)	Ouvre /proc/26404/ns/{ipc,uts,net,pid,mnt}, rejoint chacun via setns(), effectue un fork() ; l'enfant démonte puis remonte /proc (ps ne voit alors que les PID de la sandbox), résout l'utilisateur via getpwuid(1001), applique initgroups/setgid puis setuid(1001) — abandonnant les capacités — et enfin execvp("ls",["ls","-la"])
4	Helper C (parent)	waitpid() récupère le code de retour (0) et la sortie standard capturée
5	Flask	Insère une ligne Commande (texte, sortie, est_reussie=True) puis envoie ws.send({'type':"stdout",...}) suivi de ws.send({'type':"exit","ok":true})
6	Navigateur	useSandboxTerminal.onmessage ajoute l'événement reçu à l'état local ; xterm affiche la sortie à l'écran

L'ensemble de ce trajet prend typiquement **moins de 50 millisecondes** sur une machine virtuelle Linux moderne — largement sous l'exigence de latence de 200 ms fixée dans le cahier d'analyse.

11. Synthèse

L'implémentation réalisée respecte l'ensemble des engagements pris dans les cahiers d'analyse et de conception, et les complète par des choix techniques concrets :

- une architecture à trois couches strictement cloisonnées, où seul un helper C minimal et audité porte les privilèges système ;
- une authentification qui ne réinvente rien : les comptes applicatifs sont de vrais comptes Linux, vérifiés par PAM, sans aucun mot de passe stocké côté application ;
- une isolation réelle par les cinq dimensions de namespaces (PID, UTS, NET, MNT, USER), créée et rejointe par des appels système directs (clone, setns, uid_map) ;
- un terminal web temps réel appuyé sur WebSocket, avec un protocole JSON simple et une whitelist stricte en mode DEFENSE ;
- une démonstration pédagogique encadrée des vulnérabilités classiques de conteneurisation (injection shell, défaut d'isolation, absence de user namespace) et de leurs corrections respectives.

La prochaine étape naturelle du projet consiste à finaliser les tests de charge (5 sandboxes simultanées, latence sous 200 ms), le déploiement HTTPS sur le serveur dédié, et la rédaction du support de maintenance reprenant ce cycle de vie complet comme fil conducteur de la démonstration.

Annexe A — Où trouver quoi dans le code

Sujet	Fichier
Configuration / modes	flask-backend/app/config.py
Modèles SQLAlchemy	flask-backend/app/models/__init__.py
JWT + @login_required	flask-backend/app/auth/security.py
Auth PAM, register, login	flask-backend/app/auth/routes.py
CRUD sandboxes	flask-backend/app/sandboxes/routes.py
WebSocket terminal	flask-backend/app/ws/terminal.py
Wrapper vers le helper	flask-backend/app/privilege/ privilege_manager.py
Helper C (isolation namespaces)	flask-backend/helper/sandbox_helper.c
Services systemd (attack/defense)	flask-backend/systemd/*.service
Client HTTP typé (fetch + JWT)	src/lib/api.ts
Contexte React auth	src/lib/auth.tsx
Hook WebSocket terminal	src/hooks/useSandboxTerminal.ts
Pages React (dashboard, terminal...)	src/components/**

Annexe B — Glossaire

Terme	Définition
Namespace	Cloison du noyau qui isole une ressource (PID, réseau, montages, hostname, IPC, utilisateurs) pour un groupe de processus.
Capability	Sous-permission de root. Ex. CAP_SETUID = « peut changer d'UID », sans devoir être root pour le reste.
clone()	Appel système proche de fork(), mais avec des indicateurs (flags) permettant de créer de nouveaux namespaces au passage.
setns()	Permet de « rejoindre » un namespace existant en ouvrant /proc/<pid>/ns/<nom>.
PAM	Pluggable Authentication Modules — cadre Linux d'authentification (vérifie les mots de passe via /etc/shadow, gère le verrouillage, etc.).
JWT	Jeton signé (HMAC ici) que le client renvoie à chaque requête pour prouver son authentification ; contient l'identifiant utilisateur et l'UID Linux.
WebSocket	Canal bidirectionnel persistant construit sur HTTP, utilisé ici pour le terminal en temps réel.

Terme	Définition
uid_map / gid_map	Fichiers de /proc/<pid>/ où l'on déclare la correspondance « UID interne -> UID hôte » dans un user namespace.